BACKEND ↔ AI/ML INTERFACE SPECIFICATION

AI-Based Detection and Classification of Industrial Fires and Persistent Thermal Sources

Item	Details
Problem Statement ID	26162
Organization	National Technical Research Organisation (NTRO)
Interface	Node.js/Express Backend ↔ Python AI/ML Service
Version	1.0

1. Purpose

Defines the technical boundary and communication contract between the Backend division and AI/ML division so both can develop independently and integrate through a stable REST/JSON interface.

2. Responsibility Boundary

AI/ML Division

```
Raw/processed data → Feature engineering → ML model → Prediction → Python ML inference API
```

Backend Division

```
React → Node.js + Express → ML API → PostgreSQL/PostGIS
```

React should not directly call the ML service. Node.js/Express is the interface between frontend and AI/ML.

3. Communication Specification

Property	Specification
Protocol	HTTP/HTTPS
API style	REST
Data format	JSON
Backend	Node.js + Express.js
ML service	Python + FastAPI

```
React Frontend
■ HTTP
▼
Node.js + Express Backend
■ HTTP / JSON
▼
Python FastAPI ML Service
■
▼
Trained ML Model
```

4. Primary ML API

```
POST /api/v1/predict
```

The backend calls this endpoint when an AI classification is required.

5. Request Contract

```
{
"event_id": "EVT_001245",
"latitude": 26.8467,
"longitude": 80.9462,
"frp": 42.3,
"brightness": 325.6,
"confidence": 91,
"facility_distance_m": 120.5,
"facility_type": "industrial",
"land_cover_class": 50,
"historical_observation_count": 18,
"active_days": 12
}
```

Request Fields

Field	Type	Required	Description
event_id	String	Yes	Unique event identifier
latitude	Number	Yes	Event latitude
longitude	Number	Yes	Event longitude
frp	Number	Yes	Fire Radiative Power
brightness	Number	Yes	Brightness temperature
confidence	Number	Yes	FIRMS confidence
facility_distance_m	Number	Model-dependent	Distance to industrial facility
facility_type	String	Model-dependent	Associated facility type
land_cover_class	Integer	Model-dependent	WorldCover class
historical_observation_count	Integer	Model-dependent	Historical observations
active_days	Integer	Model-dependent	Number of active days

The exact feature list must be finalized by the AI/ML division. Backend must use the agreed schema and must not independently redefine ML features.

6. Response Contract

```
{
"event_id": "EVT_001245",
"prediction": {
"class": "industrial_fire",
"confidence": 0.94
},
"model": {
"name": "industrial_fire_classifier",
"version": "1.0"
}
}
```

7. Prediction Classes

```
industrial_fire
persistent_industrial_source
wildfire
agricultural_burning
mining_related
unknown
```

These are initial proposed classes. The final taxonomy must be frozen by the AI/ML division before the integration contract is finalized.

8. Error Contract

Invalid Input

```
{
"error": {
"code": "INVALID_INPUT",
"message": "Required feature 'frp' is missing"
}
}
```

Model/Service Error

```
{
"error": {
"code": "MODEL_ERROR",
"message": "Prediction service temporarily unavailable"
}
}
```

HTTP Status	Meaning
200	Prediction successful
400	Invalid request
422	Validation error
500	ML/model error
503	ML service unavailable

9. ML Service Health Check

```
GET /api/v1/health
```

```
{
"status": "healthy",
"model": "industrial_fire_classifier",
"model_version": "1.0"
}
```

10. End-to-End Flow

```
1. React requests an event.
2. Express retrieves the event from PostgreSQL/PostGIS.
3. Express prepares the ML request.
4. Express calls POST /api/v1/predict.
5. FastAPI validates the request.
6. ML preprocessing is applied.
7. The trained model generates a prediction.
8. FastAPI returns prediction and model metadata.
9. Express stores/returns the prediction.
10. React displays the result.
```

```
React → Express → PostgreSQL/PostGIS → Feature Mapping → FastAPI → ML Model → Prediction →
Express → React
```

11. Repository Boundary

```
project/
■■■ backend/ ← Division 2
■ ■■■ routes/
■ ■■■ controllers/
■ ■■■ services/
■ ■■■ ml-client/
■■■ ml-service/ ← Division 1
■ ■■■ app/
■ ■ ■■■ main.py
■ ■ ■■■ routes/
■ ■ ■■■ schemas/
■ ■■■ model/
■ ■■■ preprocessing/
■ ■■■ inference/
■■■ docs/
```

```
■■■ ML_API_CONTRACT.md
```

The backend ml-client only calls the ML API. It does not contain the ML model or training/preprocessing implementation.

12. Data Exchange Principle

```
FIRMS + OSM + WorldCover + Historical Data
↓
Prepared Features
↓
ML API
↓
Prediction
```

Do not send raw FIRMS/OSM or large satellite files through the prediction endpoint for every request. Exchange the features required by the final model.

If the final system requires image-based Sentinel-2 inference, a separate POST /api/v1/predict-image endpoint may be introduced later.

13. Contract to Freeze Before Coding

```
docs/ML_API_CONTRACT.md
```

Both divisions must agree on:

```
1. Endpoint
2. HTTP method
3. Request fields
4. Field names
5. Data types
6. Required/optional fields
7. Response structure
8. Classification labels
9. Error format
10. Model version
```

14. Integration Rules

```
Rule 1: Backend does not implement ML logic.
Rule 2: ML service does not implement frontend/UI logic.
Rule 3: React does not call the ML service directly.
Rule 4: ML API contract is the shared interface.
Rule 5: Schema changes require agreement by both divisions.
Rule 6: Model version is returned with predictions.
Rule 7: Raw datasets are not sent through the prediction endpoint.
```

15. Ownership Summary

Component	Owner
Data preparation	AI/ML Division
Feature engineering	AI/ML Division
ML model	AI/ML Division
FastAPI inference service	AI/ML Division
ML API contract	Shared
ML client	Backend Division
PostgreSQL/PostGIS	Backend Division
Express APIs	Backend Division
React application	Backend Division
GIS dashboard	Backend Division